



Bachelor Informatik/Wirtschaftsinformatik

Integrationsprojekt Kryptographie Teil 2

Spezifikationsdokument

Hannover, 8. Juni 2026

Sommerquartal 2026

Nina Neumann, Johanna Amini, Amirreza Ahmadi Darani, Xuan-Loc Tran, Franz Müller, Tammo Lütten,
Jannes Breuer, Bjarne Möller, Darko Marjanovic, Oliver Knöbl, Rafael Ulmer

Anmerkung

Die vorliegende Arbeit wurde im Rahmen eines wissenschaftlichen Projekts an der Fachhochschule für die Wirtschaft (FHDW) Hannover erstellt. Alle in dieser Arbeit verwendeten Informationen, Daten und Ergebnisse sind ausschließlich für akademische Zwecke bestimmt. Die Autoren übernehmen keine Verantwortung für die Richtigkeit, Vollständigkeit oder Aktualität der in dieser Arbeit enthaltenen Informationen. Jegliche Nutzung der Inhalte dieser Arbeit erfolgt auf eigenes Risiko.

Erklärung der Selbstständigkeit

Hiermit versichern wir, dass wir das vorliegende Spezifikationsdokument selbständig verfasst und keine anderen als die angegebenen Quellen verwendet haben. Alle Textstellen und andere Inhalte wie Bilder, Quellcode oder Daten, die wörtlich oder sinngemäß aus anderen Quellen entnommen sind, haben wir eindeutig mit korrekten Quellenangaben versehen. Über Zitierrichtlinien sind wir schriftlich informiert worden. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form, auch nicht in Teilen, keiner anderen Prüfungsbehörde vorgelegt.

Falls wir während der Erstellung dieser Arbeit generative KI oder andere KI-gestützte Technologien verwendet haben, die über grundlegende Werkzeuge für Übersetzungen und zur Überprüfung von Grammatik oder Rechtschreibung hinausgehen, erklären wir, nach der Nutzung dieser Tools den Inhalt unseres Spezifikationsdokuments überprüft und bearbeitet zu haben und übernehmen die volle Verantwortung für die diesbezüglichen Ausführungen. Eine Verwendung solcher Werkzeuge haben wir in unserer Arbeit dokumentiert (bspw. im Abschnitt „Struktur der Arbeit, Methodik“ oder durch Erläuterungen zur Nutzung im Anhang).

Inhaltsverzeichnis

1	Einleitung	3
2	Querschnittssektionen	4
2.1	Architektur-Übersicht	4
2.2	Gateway und Routing	5
2.3	Key Lifecycle	5
2.4	Serialisierung	5
2.5	Build, Deployment, Tests	5
3	Client	8
4	Use-Cases	9
4.1	Encrypted-Key-Value-Store	9
4.2	Encrypted-Age-Verification	10
4.3	Encrypted-Voting-&-Polling	17
4.4	Sealed-Bid-Auction	18
4.5	Encrypted-Statistics-Service	23
4.6	Encrypted-Genomics	29
4.7	Encrypted-Image-Processing	30
4.8	Encrypted-Leaderboard	31
4.9	Encrypted-Program-Execution	32

1 Einleitung

2 Querschnittssectionen

2.1 Architektur-Übersicht

Gesamtarchitektur: Das System folgt einer Microservice-Architektur. Jeder Use Case (UC) wird als eigenständiger Service umgesetzt. Dadurch erhalten die einzelnen UCs getrennte Lebenszyklen, unabhängige Deployments und eine klare fachliche Verantwortlichkeit. Zusätzlich wird eine Isolation auf Prozessebene erreicht, sodass Fehler oder Abstürze eines Services keine Auswirkungen auf andere UCs haben.

Insbesondere im Kontext von Fully Homomorphic Encryption (FHE) ist diese Trennung vorteilhaft, da jeder Service seinen eigenen ServerKey im Arbeitsspeicher hält. In einer monolithischen Architektur würden sich die Speichieranforderungen aller UCs gegenseitig beeinflussen.

Die Services sind als Rust-Anwendungen implementiert und verwenden das Framework Axum für die Bereitstellung von HTTP-APIs. Wiederkehrende Funktionalitäten wie Health-Endpunkte, Prometheus-Metriken, Tracing und OpenAPI-Dokumentation werden über gemeinsame Shared Crates bereitgestellt.

Komponenten: Die Architektur besteht aus folgenden Hauptkomponenten:

Komponente	Technologie	Verantwortung
Frontend	Angular + TypeScript + TFHE.js	Benutzeroberfläche, clientseitige FHE-Operationen, Schlüsselgenerierung
API Gateway	Traefik	Routing eingehender Requests, TLS-Terminierung
UC-Service 1-9	Rust + Axum	Fachlogik des jeweiligen Use Cases
Redis	Redis	Speicherung von Session-Daten und Schlüsseln
Kubernetes (k3s)	Kubernetes	Orchestrierung und Skalierung
Observability Stack	Prometheus, Grafana, Tempo	Monitoring, Metriken und Tracing

FHE-Grenze: Folgende Komponenten verarbeiten FHE-spezifische Datentypen:

- Frontend (TFHE.js)
- UC-Services (Rust + tfhe-rs)
- Redis (gespeicherte Schlüsselmaterialien)

Folgende Komponenten kennen keine FHE-internen Datentypen:

- Traefik Gateway
- Kubernetes
- Observability-Komponenten

2.2 Gateway und Routing

Als API-Gateway wird Traefik eingesetzt. Traefik unterstützt die Kubernetes Gateway API und übernimmt das Routing eingehender HTTP-Anfragen auf die jeweiligen UC-Services.

Routing: Das Routing erfolgt pfadbasiert. Requests werden anhand ihres URL-Pfades an den zuständigen Microservice weitergeleitet.

Rate Limiting: Traefik bietet technische Unterstützung für Rate Limiting über Middleware.

Sichtbarkeit von Daten im Gateway: Da Traefik ausschließlich als Gateway fungiert und keine FHE-Bibliotheken verwendet, verarbeitet das Gateway lediglich die eingehenden HTTP-Anfragen und die darin enthaltenen Request-Payloads.

2.3 Key Lifecycle

Das Frontend verwendet TFHE.js und führt FHE-bezogene Operationen clientseitig aus.

Redis wird zur Speicherung von Schlüsseln verwendet. Die Schlüssel werden mit einer Time-to-Live (TTL) abgelegt.

Speicherung: Schlüssel werden in Redis gespeichert und mit einer TTL versehen.

Isolation:

- Wie die Isolation zwischen Sessions umgesetzt wird
- Wie Mandanten- oder Benutzertrennung erfolgt

2.4 Serialisierung

Für die APIs werden Rust-Datenstrukturen verwendet, die gleichzeitig zur automatischen OpenAPI-Generierung genutzt werden.

API-Schnittstellen: Die OpenAPI-Spezifikation wird automatisch mit `aide` und `schemars` aus den Rust-Datentypen erzeugt. Jeder Service stellt seine API-Dokumentation unter `/docs` bereit.

2.5 Build, Deployment, Tests

Build-Pipeline: Das Projekt wird als Cargo-Workspace und Monorepo organisiert.

Für die Containerisierung wird ein eigenes Docker-Image pro UC-Service verwendet. Die Images werden mittels Multi-Stage-Builds und `cargo-chef` erzeugt, um Abhängigkeiten zwischenspeichern und Buildzeiten zu reduzieren.

Zur Optimierung der FHE-Leistung wird die `tfhe`-Crate bereits im Entwicklungsprofil mit `opt-level=3` kompiliert.

Für produktive Builds wird zusätzlich eine CPU-Optimierung über `target-cpu=znver5` genutzt.

CI/CD: Die CI/CD-Pipeline basiert auf GitHub Actions.

Wesentliche Bestandteile:

- Pfadbasierte Change Detection mittels `dorny/paths-filter`

- Clippy als verpflichtendes Qualitäts-Gate (-D warnings)
- Automatischer Patch-Version-Bump nach Merge auf main
- Veröffentlichung der Container-Images in GHCR
- Automatische Aktualisierung der Helm-Konfiguration
- Deployment über ArgoCD nach dem GitOps-Prinzip

Deployment: Das Deployment erfolgt auf einem k3s-Cluster.

Verwendete Infrastrukturkomponenten:

- Traefik (Gateway)
- ArgoCD (GitOps)
- KEDA (Scale-to-Zero)
- Redis
- Prometheus
- Grafana
- Tempo

Helm-Charts werden zur Verwaltung aller Kubernetes-Ressourcen verwendet.

Teststrategie: Die Teststrategie umfasst:

- Unit-Tests: Tests einzelner Komponenten und Funktionen innerhalb der Services.
- Integrationstests: Tests der Endpunkte und des Zusammenspiels mehrerer Komponenten. Die Integrationstests werden im Release-Modus ausgeführt, um realistische Laufzeiten für FHE-Operationen zu gewährleisten.
- FHE-Korrektheitstests: FHE-Ergebnisse werden gegen Klartextreferenzen validiert.

Testausführung:

- cargo nextest als Test-Runner
- cargo llvm-cov zur Ermittlung der Testabdeckung
- Ausführung mit --test-threads=1, um Speicherprobleme durch parallele ServerKey-Dekompression zu vermeiden

Testabdeckung: Die Testabdeckung wird über cargo llvm-cov ermittelt.

Hinzufügen eines neuen Services:

1. Neuer Rust-Service als Crate im Cargo-Workspace anlegen.
2. Shared Crates als Dependencies einbinden.

3. Dockerfile und Helm-Chart ergänzen.
4. Routing-Konfiguration im Gateway erweitern.
5. GitHub-Action-Pipeline aktualisieren.
6. Deployment über ArgoCD automatisch ausrollen.

3 Client

4 Use-Cases

4.1 Encrypted-Key-Value-Store

4.1.1 Funktionsbeschreibung

4.1.2 OpenAPI-Schnittstelle

4.1.3 Trust- und Threat-Model

4.1.4 FHE-Designentscheidungen

4.1.5 Komplexität der eigenen Algorithmen

4.1.6 Performance-Messung

4.1.7 Limitationen

4.2 Encrypted-Age-Verification

4.2.1 Funktionsbeschreibung

Der Use Case Encrypted Age Verification löst das Problem, das Alter einer Person serverseitig zu prüfen, ohne dass der Server den tatsächlichen Alterswert jemals im Klartext zu Gesicht bekommt. Das Ergebnis der Prüfung (volljährig: ja/nein) wird ebenfalls verschlüsselt zurückgegeben - der Server kennt weder Eingabe noch Ausgabe.

Das Alter wird bereits auf dem Client mit dem ClientKey verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend führt die Altersüberprüfung mittels Fully Homomorphic Encryption (TFHE) durch, ohne den zugrunde liegenden Alterswert entschlüsseln zu können. Die Entschlüsselung des Ergebnisses ist ausschließlich durch den Besitzer des ClientKeys möglich.

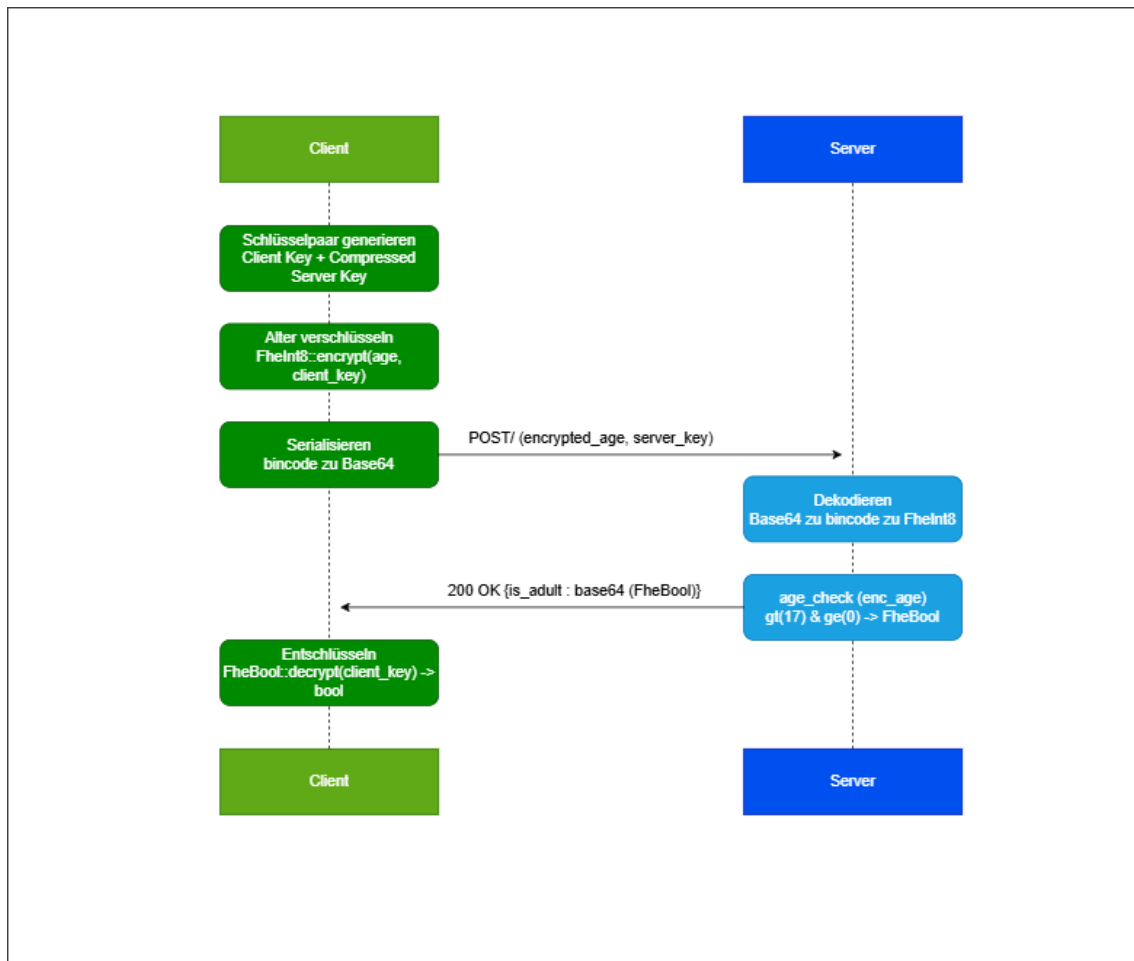
Akteure:

- Client: Generiert das TFHE-Schlüsselpaar, verschlüsselt das Alter mit dem ClientKey, sendet `encrypted_age` und `server_key` an den Server, empfängt das verschlüsselte Ergebnis und entschlüsselt es lokal. Der Client besitzt den ClientKey und ist der einzige Akteur, der das Ergebnis entschlüsseln kann.
- Backend (Server): Empfängt `encrypted_age` und `CompressedServerKey`, setzt den ServerKey, führt die homomorphe Altersüberprüfung (`age_check`) durch und gibt das verschlüsselte Ergebnis (`FheBool`) zurück. Der Server sieht zu keinem Zeitpunkt das Alter oder das Ergebnis im Klartext.

Lebenszyklus einer Session: Da dieser Use Case vollständig zustandslos ist, gibt es keine persistente Session. Der gesamte Ablauf findet innerhalb eines einzelnen HTTP-Requests statt:

1. Vorbereitung (Client): Client generiert ClientKey und CompressedServerKey lokal. Das Alter wird als `FheInt8` mit dem ClientKey verschlüsselt. Beide Werte werden bincode-serialisiert und base64-kodiert.
2. Request: Client sendet POST / mit `encrypted_age` und `server_key` als JSON-Body.
3. Verarbeitung (Server): Server dekodiert und deserialisiert beide Felder, setzt den ServerKey im globalen Thread-Kontext und führt `age_check` aus: `enc_age.gt(17) & enc_age.ge(0)`.
4. Response: Server serialisiert das `FheBool`-Ergebnis, base64-kodiert es und gibt es als `is_adult` zurück.
5. Auswertung (Client): Client dekodiert und deserialisiert `is_adult`, entschlüsselt das `FheBool` mit dem ClientKey und erhält das boolesche Ergebnis.

Verhaltensdiagramm:



4.2.2 OpenAPI-Schnittstelle

Der Service stellt eine einzelne verschlüsselte Verifikations-API bereit. Die OpenAPI-Definition wird automatisch generiert und ist unter `/openapi.json` sowie `/docs` (Swagger UI) erreichbar.

Im gesamten Service wird der ServerKey als `CompressedServerKey` übertragen (bincode-serialisiert, base64-kodiert).

POST /: Führt eine verschlüsselte Altersverifikation durch.

Request:

```
{
  "encrypted_age": "string",
  "server_key": "string"
}
```

Response 200:

```
{
  "is_adult": "string"
}
```

`is_adult` ist ein base64-kodierter, bincode-serialisierter `FheBool` - `true` wenn `Alter ≥ 18` und `≥ 0`.

Fehlercodes:

- 400 - Ungültiges Base64 oder beschädigtes bincode in server_key oder encrypted_age
- 500 - Serialisierungsfehler beim Kodieren des Ergebnisses

Body-Limit: 2 GiB (notwendig wegen der Größe des CompressedServerKey)

4.2.3 Trust- und Threat-Model

	Server klar	Server verschlüsselt	Nur Client
Alter (numerischer Wert)		X	
Ergebnis (volljährig: ja/nein)		X	
ServerKey / CompressedServerKey	X		X
ClientKey			X

Analyse: Beobachtbare Metadaten: TFHE schützt ausschließlich den Inhalt des Alters und des Ergebnisses. Für den Server bleiben weiterhin verschiedene Metadaten sichtbar:

- Zeitpunkte und Häufigkeit der Anfragen
- IP-Adresse des anfragenden Clients
- Charakteristische Payload-Größe (80 MB), die auf TFHE-Schlüsselübertragung schließen lässt
- Anzahl der Anfragen pro IP

Da age_check eine datenunabhängige Berechnung ist (keine Branches auf verschlüsselten Werten), gibt es kein Timing-Side-Channel über die Rechendauer. Ein Server-Operator kann aus den Metadaten allenfalls Nutzungshäufigkeit ableiten, nicht jedoch das Alter einzelner Nutzer.

Restvertrauen in den Server: Der Server kann zwar das Alter nicht entschlüsseln, muss jedoch weiterhin als korrekter Ausführer der Verifikationslogik vertrauenswürdig sein. Insbesondere wird angenommen, dass der Server:

- age_check korrekt und unverändert ausführt (kein Austausch gegen eine Funktion, die immer true zurückgibt)
- Das korrekte FheBool-Ergebnis zurücksendet und es nicht durch einen manipulierten Ciphertext ersetzt
- Den ServerKey nicht persistiert oder weitergibt

TFHE reduziert somit die Vertrauensabhängigkeit hinsichtlich der Inhaltsvertraulichkeit, ersetzt jedoch kein vollständig vertrauensloses Protokoll.

Annahmen außerhalb von TFHE: Die Sicherheitsbetrachtung basiert auf folgenden Annahmen:

- Die Kommunikation zwischen Client und Server erfolgt über TLS.

- Der Client führt Verschlüsselung und Entschlüsselung korrekt aus.
- Die verwendete TFHE-rs-Bibliothek wird als kryptographisch korrekt implementierte Black Box betrachtet.
- Es gibt keine Authentifizierung am Endpunkt: Jeder, der einen gültigen `CompressedServerKey` besitzt, kann Anfragen stellen.

Schutzversprechen: Durch den Einsatz von TFHE wird garantiert:

- Der Server kann das Alter des Nutzers nicht im Klartext lesen.
- Der Server kann nicht feststellen, ob das Ergebnis `true` oder `false` ist.
- Das Alter wird ausschließlich verschlüsselt verarbeitet.

Nicht garantiert werden:

- Schutz vor einem Server, der `age_check` durch eine manipulierte Funktion ersetzt
- Schutz vor Traffic-Analyse (Payload-Größe, Timing)
- Authentizität des Clients (kein ZKP, dass der Client den `ServerKey` korrekt erzeugt hat)

Konkret bedeutet das: Der Server kennt nicht das Alter des Nutzers und kann nicht feststellen, ob das Ergebnis positiv oder negativ ausgefallen ist. Sichtbar bleiben ausschließlich Zeitpunkt, Herkunft und Häufigkeit der Anfragen.

4.2.4 FHE-Designentscheidungen

Verwendete TFHE-rs-Typen: Für das Alter wird `FheInt8` (vorzeichenbehaftetes 8-Bit-Integer, Wertebereich -128 bis 127) verwendet. Diese Wahl ergibt sich aus zwei Gründen:

Altersangaben in Jahren liegen typischerweise im Bereich 0-127, also weit innerhalb von `i8`. `FheUInt8` wäre für den positiven Ast ausreichend, aber `FheInt8` erlaubt es, negative Eingaben explizit als ungültig zu erkennen und abzufangen (s. `is_positive-Check`). Zudem unterstützt `FheInt8` `gt` und `ge` mit vorzeichenbehafteter Ganzzahlssemantik, was den Negativen-Grenzwert-Test (`ge(0)`) korrekt macht.

Das Ergebnis ist ein `FheBool` (verschlüsseltes Bit), was dem binären Charakter der Frage (volljährig: ja/nein) entspricht.

Verwendete homomorphe Operationen: Der Server führt ausschließlich zwei Vergleiche und eine boolesche Verknüpfung aus. Andere homomorphe Operationen werden nicht benötigt, dadurch bleibt die serverseitige Auswertung auf die minimal nötige Anzahl FHE-Operationen beschränkt.

Operation	Verwendung
<code>FheInt8::gt</code>	<code>enc_age > 17</code> → Alter ≥ 18
<code>FheInt8::ge</code>	<code>enc_age > 0</code> → kein negativer Wert
<code>FheBool::bitand</code>	Verknüpfung beider Bedingungen

Verworfen Alternativen: `FheUInt8` statt `FheInt8`

Wäre für rein positive Eingaben ausreichend. Verworfen, weil damit keine sinnvolle Behandlung negativer Eingaben möglich ist - `FheUInt8` interpretiert `-1` als `255`, was den Negativen-Grenzwert-Test (`age_check(-1)` $\rightarrow$ `false`) unmöglich machen würde.

Schwellwert als verschlüsselter Parameter

Denkbar wäre, den Grenzwert (18) ebenfalls als `FheInt8` zu übergeben, um ihn serverseitig variabel zu halten. Verworfen, weil der Schwellwert in diesem Use Case keine schützenswerte Information ist und eine feste Konstante die Implementierung erheblich vereinfacht.

`FheInt32` oder größere Typen

Größere Bitbreiten würden höhere Latenzen pro homomorpher Operation verursachen, ohne Mehrwert - Altersangaben benötigen keine mehr als 8 Bit.

4.2.5 Komplexität der eigenen Algorithmen

Da der Use Case ausschließlich aus einer festen Anzahl homomorpher Operationen auf einem einzelnen Ciphertext besteht, gibt es keine parametrisierten Eingabegrößen. Die Komplexität aller Funktionen ist konstant.

Funktion	Zeitkomplexität	Platzkomplexität
<code>decode_server_key</code>	$O(1)$	$O(1)$
<code>decode_encrypted_age</code>	$O(1)$	$O(1)$
<code>age_check</code>	$O(1)$	$O(1)$
<code>encode_result</code>	$O(1)$	$O(1)$
<code>verify_age</code> (gesamt)	$O(1)$	$O(1)$

`verify_age` - $O(1)$, $O(1)$

Die Funktion führt genau zwei Vergleiche (`gt(17)`, `ge(0)`) und eine AND-Verknüpfung (`&`) auf einem `FheInt8`-Wert durch. Alle drei sind Operationen fester Bitbreite (8 Bit) auf einem einzelnen Ciphertext - unabhängig von jeder Eingabegröße. Gemäß der Konvention, homomorphe Operationen als $O(1)$ zu zählen, ist `age_check` $\in O(1)$.

Die De- und Serialisierungsschritte (`bincode`, `base64`) operieren auf Byte-Arrays fester Länge (durch die TFHE-rs-Typen bestimmt) und sind ebenfalls $O(1)$ bezüglich fachlicher Parameter.

4.2.6 Performance-Messung

Mess-Setup & Methodik

Die Performance- und Stresstests wurden auf einem virtuellen KVM-Server von Netcup durchgeführt. Die Last wurde extern mittels `k6` von einer lokalen Windows-Maschine über das Internet injiziert.

Es wurde ein einziger relevanter Endpunkt getestet: `POST /`. Alle anderen Endpunkte (`/health`, `/docs`) stellen nur Grundrauschen dar und wurden nicht gemessen.

Die gemessene Gesamtlatenz setzt sich daher aus drei Anteilen zusammen:

1. Netzwerkübertragung des 80 MB ServerKey (dominanter Anteil bei Remote-Messung)
2. `CompressedServerKey::decompress()` - rechenintensive Dekomprimierung
3. `age_check()` - zwei FHE-Vergleiche + AND-Verknüpfung

Die gemessenen Latenzen spiegeln den realistischen End-to-End-Overhead des zustandslosen Designs wider.

- Tool: k6 v2.0.0
- TFHE: ConfigBuilder::default()
- Datum: 05.06.2026
- Server: Netcup KVM

Test 1 - Baseline (1 VU, 10 sequentielle Requests) (05.06.2026)

Metrik	Wert
p50	13,65 s
p90	29,26 s
p95	43,51 s
Maximum	60,00 s
Fehlerrate	0%
Durchsatz	0,05 req/s

Fazit von Test 1:

Bereits bei einem einzelnen sequentiellen Client ist die Latenz hoch. Der dominierende Faktor ist die Übertragung des 80 MB ServerKey über das Internet sowie dessen Dekomprimierung. Die hohe Varianz zwischen p50 (13,65 s) und p95 (43,51 s) deutet darauf hin, dass Netzwerkschwankungen einen erheblichen Einfluss haben.

Test 2 - Stresstest (ramping bis 10 VUs) (05.06.2026, 3 Durchläufe)

Der Test wurde dreimal unabhängig voneinander ausgeführt. Die Tabelle zeigt Mittelwerte sowie die beobachtete Spanne zur Einordnung der Varianz.

Metrik	Lauf 1	Lauf 2	Lauf 3	Mittelwert
p50	13,65 s	13,70 s	13,59 s	13,65 s
p90	37,23 s	39,76 s	27,02 s	34,67 s
p95	43,51 s	52,74 s	30,69 s	42,31 s
Maximum	60,00 s	54,56 s	42,78 s	52,45 s
Fehlerrate	30 %	26 %	35 %	30 %
Durchsatz	0,05 req/s	0,05 req/s	0,05 req/s	0,05 req/s

Fazit von Test 2:

Unter paralleler Last liegt die Fehlerrate im Mittel bei 30%. Die Fehler sind ausschließlich HTTP 499 (Client Closed Request) und HTTP 504 (Gateway Timeout) - der vorgelagerte Nginx-Proxy trennt Verbindungen nach 60 s, bevor der Server die Verarbeitung abschließen kann. Der Server selbst lief in allen drei Durchläufen vollständig stabil und verarbeitete alle Anfragen ohne eigene Fehler.

Der p50 ist mit 13,6 s über alle Läufe sehr stabil - das ist die reine Verarbeitungszeit eines einzelnen Requests ohne Mutex-Wartezeit. Die hohe Varianz bei p95 (30-52 s) und der Fehlerrate (26-35%) ist dagegen direkt auf Netzwerkschwankungen bei der Übertragung des 80 MB großen ServerKey zurückzuführen. Je nach Netzwerklast zum Messzeitpunkt verschiebt sich der Anteil der Requests, die den Proxy-Timeout überschreiten.

Die Throughput-Grenze liegt bei einem parallelen Request. Jeder weitere gleichzeitige Request verlängert die Wartezeit linear, da `tfhe::set_server_key` einen globalen Thread-Kontext

setzt und die gesamte Verarbeitung (Übertragung + Dekomprimierung + FHE) serialisiert wird. Ab 2 VUs überschreitet p95 den Proxy-Timeout von 60 s regelmäßig.

4.2.7 Limitationen

- Der Use Case ist vollständig zustandslos. Es gibt keine Session, keine Audit-Log und keine Möglichkeit, Ergebnisse serverseitig zu speichern oder abzurufen.
- Der `CompressedServerKey` (80 MB) wird bei jedem einzelnen Request vom Client mitgeschickt, deserialisiert und dekomprimiert. Dieses Design ist eine direkte Konsequenz der Zustandslosigkeit: Da jeder Client sein eigenes Schlüsselpaar generiert, kann kein globaler `ServerKey` serverseitig gespeichert werden, ohne das Sicherheitsmodell zu brechen. Ein gespeicherter `ServerKey` eines anderen Clients würde es diesem ermöglichen, fremde Ergebnisse zu entschlüsseln. Die Folge ist, dass Netzwerkübertragung und Dekomprimierung die Latenz dominieren und ein produktiver Einsatz über das Internet praktisch nicht skaliert.
- Der Server prüft nicht, ob ein `encrypted_age`-Ciphertext bereits zuvor verwendet wurde. Ein Angreifer, der einen Ciphertext abfängt, kann ihn beliebig oft einreichen.
- Der Client übergibt den `CompressedServerKey` selbst. Ein bössartiger Client könnte einen manipulierten Key einreichen. In einer produktiven Umgebung müsste der `ServerKey` serverseitig fest hinterlegt sein.
- Der Client könnte einen beliebigen `FheInt8`-Wert übermitteln - der Server kann nicht verifizieren, dass die verschlüsselte Eingabe tatsächlich ein Alter darstellt oder aus einer vertrauenswürdigen Quelle stammt (kein Zero-Knowledge-Beweis).
- Maximal darstellbarer Alterswert ist 127 Jahre (`i8::MAX`). Dies ist für den Anwendungsfall ausreichend, aber die Wahl von `FheInt8` schließt größere Ganzzahlen strukturell aus.
- Es gibt keine Authentifizierung am Endpunkt. Jeder, der die API kennt, kann Anfragen stellen. Ein Gateway-Layer (z. B. API-Key, mTLS) ist in der aktuellen Implementierung nicht vorhanden.
- Durch den globalen `set_server_key`-Aufruf ist echte Parallelverarbeitung nicht möglich. Der Durchsatz skaliert nicht mit der Anzahl der CPU-Kerne.

4.3 Encrypted-Voting-&-Polling

4.3.1 Funktionsbeschreibung

4.3.2 OpenAPI-Schnittstelle

4.3.3 Trust- und Threat-Model

4.3.4 FHE-Designentscheidungen

4.3.5 Komplexität der eigenen Algorithmen

4.3.6 Performance-Messung

4.3.7 Limitationen

4.4 Sealed-Bid-Auction

4.4.1 Funktionsbeschreibung

Der Use Case Sealed-Bid Auction ermöglicht die Durchführung verdeckter Auktionen (Blin-dauctionen) zwischen mehreren Bietern. Ziel ist es, nach dem Ende der Bietphase den Höchstbietenden sowie den exakten Gewinnerbetrag zu ermitteln, ohne dass der Server zu irgendeinem Zeitpunkt die einzelnen Gebote der Teilnehmer im Klartext einsehen kann.

Hierzu werden die Gebotsbeträge bereits auf dem Client verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend verarbeitet und evaluiert die verschlüsselten Gebote mittels Fully Homomorphic Encryption (TFHE), ohne die zugrunde liegenden Klartexte kennen zu können. Die Entschlüsselung des finalen Auktionsgewinners ist ausschließlich durch den Besitzer des ClientKeys (den Auktionator) möglich.

Akteure:

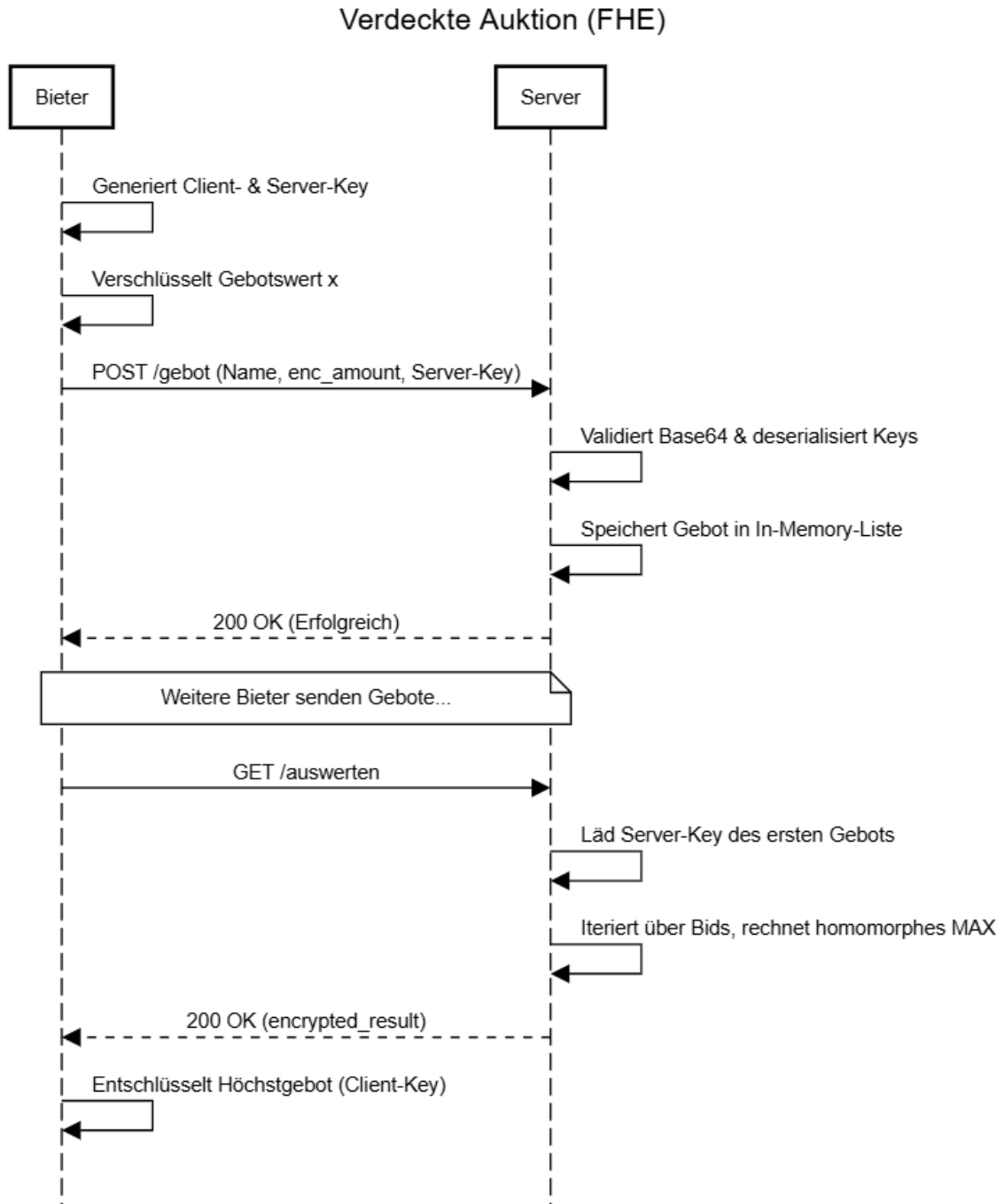
- Auktionator (Client E): Der Auktionator initialisiert das TFHE-Schlüsselpaar, erstellt die Auktionsrunde und gibt die Auktion für die Bieter frei. Er besitzt den geheimen ClientKey und ist als einziger Akteur in der Lage, das vom Server homomorph berechnete Endergebnis (den Höchstbietenden und den Gewinnbetrag) lokal zu entschlüsseln.
- Bieter (Client): Ein Bieter kann der Auktion beitreten, indem er seinen Namen und sein verdecktes Gebot abgibt. Das Gebot wird clientseitig mit dem PublicKey als FheUint32 verschlüsselt. Bieter können zu keinem Zeitpunkt die Gebote anderer Teilnehmer oder den aktuellen Zwischenstand der Auktion einsehen.
- Backend (Server): Der Server verwaltet den Zustand der Auktion, speichert die eingehenden verschlüsselten Gebote flüchtig im RAM und stellt den PublicKey für neue Bieter bereit. Auf Anfrage führt er die rechenintensive, blinde Maximumsuche über homomorphe Größenvorzeichen-Operationen und Auswahlen durch, ohne Zugriff auf Klartextinformationen zu haben.

Lebenszyklus einer Session:

1. Erstellung: Der Auktionator generiert ein TFHE-Schlüsselpaar und startet die Auktion mit einer eindeutigen UUID. Der PublicKey und der komprimierte ServerKey werden an das Backend übergeben, der ClientKey verbleibt exklusiv im lokalen Session Storage des Auktionators.
2. Gebotsabgabe: Bieter rufen das Dashboard auf. Das Frontend lädt den PublicKey vom Server. Der Bieter gibt seinen Namen ein, das Frontend verschlüsselt den Betrag als FheUint32 und sendet den Payload an /auction/gebot. Der Server speichert den Namen im Klartext und das Gebot als Ciphertext im globalen RAM (Vec<Bid>).
3. Auswertung: Sobald die Bietphase beendet ist, klickt der Auktionator on „Gewinner berechnen“. Der Server prüft, ob Gebote vorhanden sind, und durchläuft anschließend eine homomorphe Schleife, um das absolute Maximum und den zugehörigen Namen ohne Entschlüsselung zu isolieren. Das verschlüsselte Ergebnis wird für den Auktionator hinterlegt.
4. Schließen & Finalisierung: Nach der Auswertung wird die Auktion als geschlossen markiert. Es werden keine neuen Gebote mehr akzeptiert. Der Auktionator ruft das

verschlüsselte Endergebnis ab, entschlüsselt es lokal mit seinem ClientKey und zeigt den Gewinner auf dem Dashboard an.

Verhaltensdiagramm:



4.4.2 OpenAPI-Schnittstelle

Die API bietet zwei REST-Endpunkte, welche semantisch als Request/Response via JSON konzipiert sind. Die kryptographischen Payloads sind per bincode serialisiert und in Base64 codiert.

POST /gebot Empfängt ein verschlüsseltes Gebot.

- Request Body (JSON):
 - bidder_name (String): Klartextname des Bieters.
 - encrypted_amount (String): Base64-codierter tfhe::FheUint32 Payload.
 - server_key (String): Base64-codierter tfhe::CompressedServerKey.
- Response (200 OK):
 - response (String): Erfolgsmeldung.
- Fehler (400 Bad Request):
 - Tritt auf, wenn Base64 nicht decodiert werden kann oder die Deserialisierung in TFHE-Typen fehlschlägt. Rückgabe als String im Body.

GET /auswerten Wertet die gespeicherten Gebote homomorph aus.

- Response (200 OK):
 - status (String): Meta-Information über die Anzahl der ausgewerteten Gebote.
 - encrypted_result (String): Base64-codierter tfhe::FheUint32 Payload des höchsten Gebotes.
- Fehler (400 Bad Request):
 - Tritt auf, wenn die interne Gebotsliste leer ist.

4.4.3 Trust- und Threat-Model

	Server klar	Server verschlüsselt	Nur Client
Bieter-Name (bidder_name)	X		
Gebotshöhe (encrypted_amount)		X	
Max-Gebot (encrypted_result)		X	
Server-Key	X		
Client-Key			X

Metadaten-Analyse: Ein böartiger Server-Operator kann die exakte Anzahl der Gebote, die Identitäten (Namen) der Teilnehmer und die genauen Zeitpunkte der Gebotsabgaben einsehen. Da FHE-Chiffre für denselben Datentyp statische Größen aufweisen, sind Rückschlüsse auf die Höhe des Gebots durch Längenanalysen ausgeschlossen. Der Server-Operator könnte Traffic-Analysen fahren oder die Auswertung blockieren (Denial of Service).

Restvertrauen & Garantien: Es muss dem Server vertraut werden, dass er den Algorithmus korrekt anwendet. Ein manipulierender Server könnte willkürlich legitime Gebote aus der Liste entfernen (Zensur) oder das Resultat fälschen, indem er einfach das Chiffre eines bevorzugten Bieters als Ergebnis zurückschickt, ohne die homomorphe Auswertung jemals durchgeführt zu haben. Schutzversprechen: Der Server lernt unter keinen Umständen die Höhe der eingereichten Gebote und auch nicht den Gewinner-Wert.

4.4.4 FHE-Designentscheidungen

- Typen-Wahl: Für die Gebotshöhen wird `FheUInt32` verwendet. 32-Bit Unsigned Integer erlauben Gebote bis zu knapp 4,3 Milliarden, was für reale Auktionsszenarien absolut ausreichend ist. Der Typ ist deutlich performanter auszuwerten als `FheUInt64`.
- Operationen: Zur Ermittlung des Höchstgebots werden ausschließlich der numerische Vergleich `.gt()` (Greater Than) und das Multiplexing `.select()` verwendet. Mathematische Operationen wie Addition oder Multiplikation sind hier nicht erforderlich.
- Verworfenen Varianten: Die Sortierung der gesamten Liste wurde verworfen, da eine vollständige Sortierung in FHE aufgrund von datenunabhängigen Branches Komplexität aufbaut und nicht nötig ist. Stattdessen implementiert der Code ein lineares Scannen (Max-Suche), das den aktuellen Höchstwert überschreibt.

4.4.5 Komplexität der eigenen Algorithmen

Die Funktion `evaluate_encrypted_auction` iteriert über die Liste der eingegangenen Gebote. Sei n die Anzahl der abgegebenen Gebote (wobei $n \geq 1$).

Der Algorithmus überspringt das erste Element und iteriert exakt $n - 1$ mal. Pro Iteration werden exakt zwei FHE-Operationen ausgeführt:

- Ein Vergleich (Greater Than): `gt()`
- Eine Auswahl (Mux): `select()`

Die asymptotische Laufzeit im Hinblick auf eigene FHE-Operationen beträgt damit exakt:

Komplexität = $O(n)$ Der Platzbedarf während der Auswertung (zusätzlich zu den eingelesenen Chiffren) ist $O(1)$, da stets nur eine einzige Variable (`maximales_gebot`) für den Zwischenstand genutzt wird.

4.4.6 Performance-Messung

Die Performancemessung fand auf einem Netcup RS 1000 G9.5 statt, dem 2 vCores und 4 GB RAM zugewiesen wurden. Als Testdatensatz dienten Serienschlüsselungen über ein automatisiertes Benchmark-Skript.

- Latenzen: Bei 10 sequentiellen Geboten beträgt die Auswertungs-Latenz (p50) auf dem Endpoint `/auswerten` ca. 2,83 Sekunden. Bei 50 Geboten steigt p95 auf ca. 14,15 Sekunden, was der streng linearen Skalierung ($O(n)$) der homomorphen Schleife entspricht.
- Speicherbedarf: Während der Auswertung von 50 Bids werden ca. 120 MB RAM verbraucht, was primär auf das einmalige Laden des großen `CompressedServerKey` in den Arbeitsspeicher zurückzuführen ist.
- Durchsatzgrenzen: Wenn parallel mehrere `GET /auswerten`-Requests eintreffen (ab ca. 1 bis 2 Requests / Sekunde), gerät der Server ans CPU-Limit voreingestellter Threadpools. Da der globale Zustand über ein `state.lock()` (Mutex) während der gesamten FHE-Berechnung blockiert wird, entsteht bei parallelen Zugriffen ein massiver

Verarbeitungstau, der Timeouts (>30s) bei den Clients auslöst. Die Gebotsannahme (POST /gebot) ist hingegen nicht durch FHE-Operationen gebunden (die Ciphertexte werden lediglich als Bytes entgegengenommen und im RAM abgelegt) und skaliert im Bereich von Hunderten Requests pro Sekunde mühelos.

4.4.7 Limitationen

1. Keine Auflösung des Gewinners: Das System liefert ausschließlich den Betrag des höchsten Gebotes als Chiffre zurück. Der Server kann fachlich nicht feststellen, hinter welchem bidder_name dieser Betrag steckt. Dem Bieter ist zwar das Max-Gebot nach Entschlüsselung bekannt; es existiert abseits von Off-Chain-Verifikationen jedoch aktuell kein kryptographischer Nachweis, der den Gewinner-Betrag mit dem einreichenden Namen verknüpft.
2. Single-Key Assumption (Multi-Key FHE fehlt): Die aktuelle Implementierung unterstellt im Code (`let real_sk_bytes = &liste[0].server_key_bytes;`), dass alle Bieter mit demselben gemeinsamen ClientKey verschlüsselt haben. Ein echtes verteiltes Trust-System, in dem jeder Bieter einen eigenen unabhängigen Key besitzt (Multi-Key FHE), ist nicht implementiert.
3. Flüchtiger Speicherzustand: Bids werden im Arbeitsspeicher über `static BIDS: Mutex<Vec<Bid>>` vorgehalten. Bei Container- oder Anwendungsneustarts ist die gesamte Auktion unwiederbringlich verloren.
4. Fehlende Zugriffskontrolle: Es gibt derzeit keinen Authentifizierungsmechanismus. Jeder Client kann beliebig viele Gebote übermitteln, und jeder kann die Endauswertung triggern. Spam- und DoS-Absicherung fehlt auf Service-Ebene.

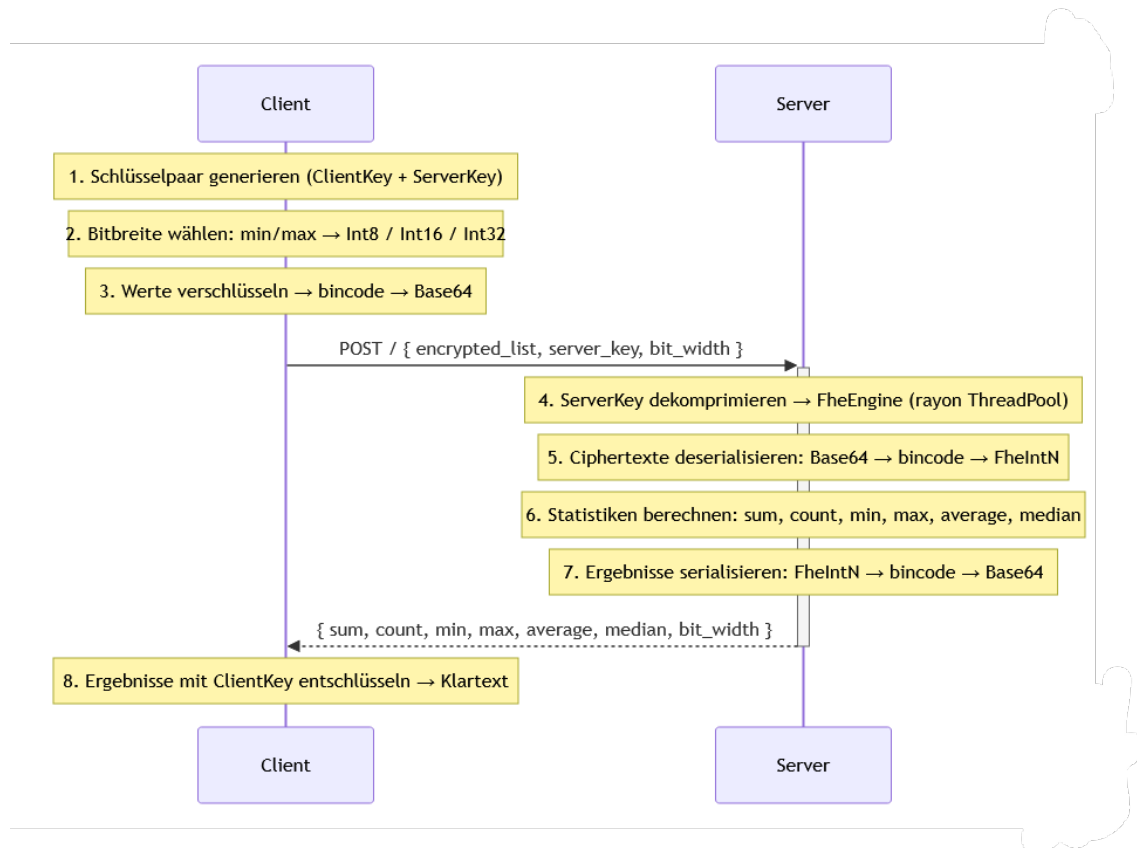
4.5 Encrypted-Statistics-Service

4.5.1 Funktionsbeschreibung

Akteure: Client (Browser, Angular-Frontend), Server (Axum/Rust-Service).

UC5 berechnet statistische Kennzahlen (Summe, Anzahl, Minimum, Maximum, Durchschnitt, Median) über eine Ganzzahlen-Liste, ohne dass der Server die Eingabewerte oder Ergebnisse jemals im Klartext sieht. Alle Berechnungen laufen vollständig auf verschlüsselten Daten.

Ablauf eines Requests (UC5 ist zustandslos — jeder Request ist in sich abgeschlossen):



4.5.2 OpenAPI-Schnittstelle

POST /: Berechnet alle Statistiken für eine verschlüsselte Ganzzahlen-Liste.

Request-Schema:

```
{
  "encrypted_list": [
    "<Base64-String>",
    "...",
  ],
  "server_key": "<Base64-String>",
  "bit_width": 8
}
```

Feld	Typ	Beschreibung
encrypted_list	string[]	Jedes Element: Base64(bincode(FheIntN)), wobei N = bit_width. Mindestens 1 Element.
server_key	string	Base64(bincode(CompressedServerKey)) - vom Client generiert, enthält kein Geheimnis.
bit_width	number	Muss 8, 16 oder 32 sein. Wird vom Client automatisch aus dem Wertebereich der Eingabe gewählt.

Response-Schema (200 OK):

```
{
  "sum":      "<Base64-String >",
  "count":    42,
  "min":      "<Base64-String >",
  "max":      "<Base64-String >",
  "average":  "<Base64-String >",
  "median":   "<Base64-String >",
  "bit_width": 8
}
```

Feld	Typ	FHE-Typ (je nach bit_width)
sum	string	Base64(bincode(FheInt16/32/64)) - eine Stufe breiter als Eingabe
count	number	Klartextzahl - dem Server schon aus dem Request bekannt
min	string	Base64(bincode(FheInt8/16/32)) - gleiche Breite wie Eingabe
max	string	Base64(bincode(FheInt8/16/32)) - gleiche Breite wie Eingabe
average	string	Base64(bincode(FheInt16/32/64)) - eine Stufe breiter als Eingabe
median	string	Base64(bincode(FheInt8/16/32)) - gleiche Breite wie Eingabe
bit_width	number	Tatsächlich verwendete Bitbreite: 8, 16 oder 32

Fehlerstatus:

Status	Bedeutung
400 Bad Request	Leere Liste, ungültiger Base64, falscher Ciphertext-Typ, ungültige bit_width
500 Internal Server Error	Serialisierungsfehler beim Zusammenstellen der Response

Weitere Endpunkte:

Endpunkt	Methode	Beschreibung
/healthz	GET	Liveness-Probe (Kubernetes)
/readyz	GET	Readiness-Probe (Kubernetes)
/version	GET	Service-Version als JSON
/metrics	GET	Prometheus-Metriken
/docs	GET	Swagger UI (OpenAPI-Dokumentation)
/openapi.json	GET	OpenAPI-Spec als JSON

4.5.3 Trust- und Threat-Model

Was am Server bekannt ist:

Datum	am Server klar	am Server verschlüsselt	nur am Client
Eingabewerte (die eigentlichen Zahlen)	X	FheInt8/16/32	Klartext vor Verschlüsselung
Anzahl der Elemente (n)	(Array-Länge im Request)	-	-
Wertebereich-Klasse	(über bit_width: Int8 → Werte in [-128, 127])	-	-
Summe	X	FheInt16/32/64	nach Entschlüsselung
Minimum	X	FheInt8/16/32	nach Entschlüsselung
Maximum	X	FheInt8/16/32	nach Entschlüsselung
Durchschnitt	X	FheInt16/32/64	nach Entschlüsselung
Median	X	FheInt8/16/32	nach Entschlüsselung
ServerKey	(aus Request)	-	-
ClientKey	X	-	verlässt den Browser nie
Anfragezeitpunkt	(HTTP-Timestamp)	-	-
Payload-Größe	(HTTP-Header)	-	-

Beobachtbare Metadaten und möglicher Missbrauch:

- Anzahl n ist vollständig sichtbar. Ein Angreifer kann daraus schließen, wie viele Werte analysiert werden (z.B. „der Client hat genau 3 Werte geschickt“ → möglicherweise 3 Messwerte eines Sensors).
- bit_width verrät die Größenordnung: bit_width=8 bedeutet, alle Werte liegen in [-128, 127]. Das schränkt den möglichen Werteraum erheblich ein - bei kleinen Eingabelisten und bekanntem Kontext könnten Werte erraten werden.
- Request-Timing ist bei bekannten n-Werten korrelierbar. Die Berechnungsdauer hängt von n und bit_width ab, nicht von den konkreten Werten - es gibt kein datenwertabhängiges Timing-Leak.
- Payload-Größe ist deterministisch aus n und bit_width ableitbar (alle FHE-Ciphertexte haben bei gleichen TFHE-Parametern konstante Größe). Kein zusätzlicher Informationsgewinn.

Restvertrauen in den Server-Operator: Der Server muss die Berechnungen korrekt ausführen. Ein böswilliger Server könnte:

- Falsche verschlüsselte Ergebnisse zurückgeben (der Client kann das nicht verifizieren - kein ZKP).
- Die Ciphertexte dauerhaft speichern (für einen hypothetischen späteren Kryptangriff).
- Den Request einfach ablehnen (Denial of Service).

Annahmen außerhalb von FHE:

- TLS zwischen Client und Server (kein Mitlesen im Transit).

- Das Angular-Frontend und der Browser sind nicht kompromittiert (ClientKey liegt im WASM-Speicher).
- TFHE-rs wird als korrekte Blackbox behandelt - keine eigene Kryptanalyse.
- Kein Auth-Gateway: jeder kann beliebige Requests an POST / schicken.

Sicherheitsgarantie: Der Server kennt die statistischen Kennzahlen der Eingabewerte nicht — er berechnet sie, ohne sie je zu sehen. Bekannt sind ihm nur die Anzahl der Werte und der grobe Wertebereich (über `bit_width`). Keine ZKP-Verifikation der Ciphertexte und keine Client-Authentifizierung — beides ist im Rahmen dieses Use Cases nicht vorgesehen.

4.5.4 FHE-Designentscheidungen

Verwendete TFHE-rs-Typen:

Eingabe-Bitbreite	Eingabe-Typ	Summe/Avg-Typ	Begründung
8	FheInt8	FheInt16	Overflow-Schutz: $n * 127$ muss in den Output-Typ passen
16	FheInt16	FheInt32	Gleiche Logik, nächste Ebene
32	FheInt32	FheInt64	Int64 deckt alle realistischen Summen ab

Auto-Bitbreiten-Wahl: Der Client bestimmt anhand von `min` und `max` der Eingabe die kleinste ausreichende Bitbreite:

- Int8 wenn $\min \geq -128$ und $\max \leq 127$
- Int16 wenn $\min \geq -32.768$ und $\max \leq 32.767$
- Int32 sonst

Der Server benötigt `bit_width` zwingend, um den richtigen FHE-Typ für die Deserialisierung zu wählen - FheInt8, FheInt16 und FheInt32 sind binär inkompatibel. Das Feld ist daher keine optionale Optimierung, sondern technisch notwendig.

Kleinere Bitbreiten reduzieren die Berechnungszeit deutlich, da die TFHE-Kosten mit der Bitbreite skalieren.

Verwendete FHE-Operationen:

Operation	Verwendet für	Kosten
CastInto	Up-Cast InputType $\rightarrow$ WiderOutputType vor Summation	günstig
Add	Summation (parallel via rayon)	moderat
FheOrd (lt/gt)	Vergleich in min/max und compare_and_swap	teuer (Bootstrapping)
IfThenElse auf Fhe-Bool	Wählt das Ergebnis homomorph aus, ohne den Wert zu kennen	moderat
Division durch Klartextskalar	Durchschnitt: $\text{sum} / (n \text{ as } i16/i32/i64)$	deutlich günstiger als FHE-Division

Verworfenen Varianten:

- Float (f32/f64): Semantisch passend für Durchschnitt, aber TFHE-rs bietet keine Float-Typen.
- Vollständig homomorphe Division: TFHE-rs hat keine generische `Div<FheInt>`-Implementierung. Division durch einen Klartextwert ist hier ausreichend, weil die Listenlänge dem Server ohnehin bekannt ist.
- Naiver sequentieller Sortieralgorithmus für Median: Bubblesort wäre $O(n^2)$ Komparatoren und vollständig sequentiell - auf FHE-Niveau unakzeptabel. Das Batchers-Netzwerk ist datenunabhängig (keine Branches auf Plaintext-Vergleichsergebnissen) und hat $O(\log^2 n)$ Tiefe.
- Partielles Sortiernetz (Median-optimiert): Ein Netz, das nur den Median-Index isoliert berechnet, würde weniger Komparatoren brauchen. Nicht umgesetzt, weil der Aufwand für das UC-Projekt nicht gerechtfertigt ist.

4.5.5 Komplexität der eigenen Algorithmen

Parameter: n = Anzahl der Eingabeelemente. Jede einzelne FHE-Operation zählt als $O(1)$.
sum: Paralleles Reduce (rayon) auf n Elementen.

- Tiefe: $O(\log n)$ sequentielle Schritte (binärer Reduktionsbaum)
- Gesamtoperationen: $n-1$ Additionen
- Speicher: $O(n)$

count: $O(1)$ — nur `slice::len()`.

min / max: Identisch zu **sum** (Reduce mit Vergleich statt Addition).

- Tiefe: $O(\log n)$
- Gesamtoperationen: $n-1$ Vergleiche + $n-1$ `if_then_else`

average: $O(\log n)$ - identisch zu **sum**, plus eine $O(1)$ -Division durch Klartextskalar.

median (Batcher Odd-Even Mergesort): Das Batcher-Netzwerk für n Elemente hat (Herleitung: Knuth TAOCP Vol. 3, §5.3.4):

- Tiefe (Anzahl sequentieller Runden): $O(\log^2 n)$, exakt $\frac{1}{2} \cdot \log_2(n) \cdot (\log_2(n) + 1)$ für $n = 2^k$
- Gesamtanzahl Komparatoren: $O(n \log^2 n)$

Innerhalb einer Runde laufen alle Komparatoren parallel (rayon). Die Wandzeit entspricht der Netzwerk-Tiefe: $O(\log^2 n)$ sequentielle FHE-Runden.

Jeder Komparator besteht aus: $1 * \text{FheOrd-Vergleich} + 2 * \text{IfThenElse} = O(1)$ homomorphe Operationen.

Gesamtspeicher: $O(n)$ (der `partially_sorted`-Vektor als Arbeitspuffer).

4.5.6 Performance-Messung

Messbedingungen: lokaler Entwicklungs-PC (Windows/WSL2), Rust release build, 1 Client, keine parallelen Requests.

n	bit_width	Gesamtlatenz (Einzelmessung)	Anmerkung
6	8	18,9 s	Werte: -5, 3, 7, 5, 33, 10

Netcup-Server-Messungen (p50/p95 unter Last): ausstehend. Mess-Setup, Lastkurve und Throughput-Grenze sind für die finale Ausarbeitung noch zu erheben.

Bekannte Performance-Treiber:

- ServerKey-Dekomprimierung: `CompressedServerKey::decompress()` wird bei jedem Request neu durchgeführt - kein Caching. Das ist der größte einzelne Overhead neben der eigentlichen FHE-Berechnung.
- FheOrd-Vergleiche sind die teuersten Einzeloperationen (Bootstrapping-Kosten).
- Rayon-Parallelisierung innerhalb einer Batch-Runde skaliert mit den CPU-Kernen des Servers.
- Skalierung mit n: Verdopplung von n erhöht die Batch-Tiefe um ca. $2 \cdot \log_2 n$ Runden. Der Sprung von n=6 auf n=12 erhöht die Tiefe von 9 auf 16 Runden.

4.5.7 Limitationen

Typsystem: TFHE-rs unterstützt keine Float-Typen — alle Berechnungen sind auf i8/i16/i32 beschränkt. Der Durchschnitt ist ganzzahlig — Nachkommastellen werden abgeschnitten ($[1, 2] \rightarrow 1$, nicht 1,5).

Privacy-Einschränkungen: Die Listenlänge ist aus dem Request direkt ablesbar; `bit_width` verrät zusätzlich die Größenordnung der Werte (`bit_width=8` → alle Werte in $[-128, 127]$). Beides lässt sich durch Padding bzw. uniforme Bitbreite abmildern - für diesen Use Case nicht umgesetzt, da der Mehraufwand die Performance deutlich verschlechtern würde.

Keine Verifikation der Eingaben: Der Server kann nicht prüfen, ob der Client korrekte FHE-Ciphertexte schickt - kein ZKP, keine Authentifizierung. Bei öffentlicher Erreichbarkeit sind Missbrauch durch Masse-Requests (teurer Rechenaufwand) und manipulierte Ciphertexte möglich.

Performance: Kein Session-Caching - `CompressedServerKey::decompress()` läuft bei jedem Request neu. Bei $n > 20$ mit Int32 übersteigt die Rechenzeit mehrere Minuten; der Service ist für kleine Listen ($n \leq 15$) konzipiert. Der Batch-Algorithmus sortiert außerdem die gesamte Liste, statt nur den Median-Index zu isolieren.

4.6 Encrypted-Genomics

4.6.1 Funktionsbeschreibung

4.6.2 OpenAPI-Schnittstelle

4.6.3 Trust- und Threat-Model

4.6.4 FHE-Designentscheidungen

4.6.5 Komplexität der eigenen Algorithmen

4.6.6 Performance-Messung

4.6.7 Limitationen

4.7 Encrypted-Image-Processing

4.7.1 Funktionsbeschreibung

4.7.2 OpenAPI-Schnittstelle

4.7.3 Trust- und Threat-Model

4.7.4 FHE-Designentscheidungen

4.7.5 Komplexität der eigenen Algorithmen

4.7.6 Performance-Messung

4.7.7 Limitationen

4.8 Encrypted-Leaderboard

4.8.1 Funktionsbeschreibung

4.8.2 OpenAPI-Schnittstelle

4.8.3 Trust- und Threat-Model

4.8.4 FHE-Designentscheidungen

4.8.5 Komplexität der eigenen Algorithmen

4.8.6 Performance-Messung

4.8.7 Limitationen

4.9 Encrypted-Program-Execution

4.9.1 Funktionsbeschreibung

4.9.2 OpenAPI-Schnittstelle

4.9.3 Trust- und Threat-Model

4.9.4 FHE-Designentscheidungen

4.9.5 Komplexität der eigenen Algorithmen

4.9.6 Performance-Messung

4.9.7 Limitationen